

C R E D E N C E

CREDIT RISK INTELLIGENCE PLATFORM

Fresher Data Analyst Interview Prep

Questions a fresher-level Data Analyst interview in India will actually ask about this project — SQL, Excel/Power BI, Python, stats — with detailed answers grounded in the real code.

Calibrated for entry-level rounds — SQL and Excel/Power BI carry the most weight, ML depth is kept light.

Before the interview

This is calibrated for a FRESHER-level Data Analyst round in India — campus hiring, an analytics/IT-services firm, or an entry-level product-company role — not a credit-risk or data-science interview. That means SQL and Excel/Power BI carry the most weight, the project walkthrough matters more than deep ML theory, Python stays at the pandas-basics level, and statistics questions test whether you can explain a concept simply, not derive it. Have the two-minute pitch below memorized well enough to say naturally, not recite — everything else in this document is what happens when the interviewer starts pulling a thread from it.

The two-minute pitch

"I built an end-to-end credit risk pipeline on Lending Club's public loan data — over 640,000 real loans. I cleaned and staged the raw data in SQL, engineered a leakage-safe feature set, and trained a model to predict the probability a loan defaults. I validated it out-of-time, meaning I tested it on loans issued AFTER the training period — not a random split — because that's how it would actually be used in the real world, scoring future applicants. On top of that I built a 5-page React dashboard and a Power BI star-schema model presenting portfolio risk, borrower-level detail, and a policy simulator. Every number in the dashboard traces back to a real pipeline output, and I was deliberate about separating what's real pipeline output from what's illustrative sample data, and labelling that clearly."

Say it, don't read it. The interviewer's next question is almost always "tell me more about [the one part they found interesting]" — so after you say this, stop talking and let them choose the thread.

SECTION 1

Project deep-dive

These start almost every fresher round. They're testing whether you actually understand your own project, not whether you memorized a script.

Q1. Walk me through this project, start to finish.

Start with the two-minute pitch, then be ready to go one level deeper wherever they pull the thread. The honest structure to have in your head: (1) raw data comes in as one big CSV, (2) it gets cleaned and typed in SQL, (3) it gets split into three separate tables so the model can only see information that would genuinely be available before a lending decision is made, (4) a model is trained and tested on loans from a later time period than it was trained on, (5) the model's output feeds an expected-loss calculation and a policy simulator, and (6) all of that gets presented in a dashboard. Don't try to explain all six in equal depth up front — say the shape of it, then let them ask.

Tip: Practice saying this out loud, timed, at least five times before the interview. It should take under 90 seconds unprompted.

Q2. Why did you pick this dataset / this problem?

Two honest reasons work well: it's real, public data at a meaningful scale (not a toy 500-row CSV), and it's a genuine business decision a real company has to make — should we approve this loan, and how much risk is the whole portfolio carrying. That combination (real scale + real decision) is what made it worth building properly rather than just running one notebook cell of `.fit()` and stopping.

Q3. What exactly was YOUR role? Did you build this entirely alone?

Answer this honestly and specifically — this is one of the most common places freshers get caught out, because the follow-up questions will expose a vague answer fast. Be ready to say precisely which pieces you designed and wrote (for example: the SQL cleaning and feature-splitting logic, the model training and validation setup, the dashboard) versus any tooling or AI assistance you used along the way. Using tools well and being able to explain what they produced is a legitimate skill — claiming sole authorship of something you can't explain in follow-up is what actually damages you.

Q4. What was the hardest part of building this?

A strong, honest answer: making sure the model couldn't accidentally 'cheat' by seeing information it shouldn't have at prediction time. Explain it in plain terms — the lender's own internal risk grade and the loan's interest rate are basically already a risk score in disguise, so if you fed those into your OWN model, you'd just be teaching it to copy an existing answer instead of learning something new. The fix was splitting the data into three separate tables at the SQL level — one the model is allowed to read from, one used only to compare against afterward, and one used only to check what actually happened — so that kind of information couldn't reach the model by accident.

Q5. What would you do differently if you started again?

Have one specific, real answer ready rather than a vague 'I'd make it better.' Good example: 'The SQL cleaning stage has around 40 column names specific to this exact dataset hardcoded into it — if I got handed a different lender's data tomorrow with different column names, it would break immediately. I'd build a small configuration

layer that maps a new dataset's columns onto the ones my pipeline expects, instead of hardcoding them, so the same pipeline could be reused.'

Q6. What tools/technologies did you use, and why those?

SQL and DuckDB for the data cleaning and transformation layer (fast, works entirely locally, no server to set up), Python with pandas/scikit-learn for the modelling, and both a React dashboard and Power BI for presenting the results — two different audiences: a code-first interactive tool, and a familiar BI tool a business stakeholder would already know how to navigate.

SECTION 2

SQL

This is where fresher Data Analyst interviews in India spend the most time, often more than half the technical round. Know these cold, and be ready to point at a real line from your own project for each one.

Q7. What's the difference between WHERE and HAVING?

WHERE filters individual rows before any grouping happens. HAVING filters AFTER rows have been grouped and aggregated — it works on the result of an aggregate function like COUNT() or SUM(), which WHERE can't do. In this project, the staging SQL uses a WHERE clause to drop rows before anything is grouped: ``WHERE loan_status IS NOT NULL AND loan_status NOT LIKE 'Does not meet the credit policy%``. A HAVING example would be something like keeping only groups with more than 500 loans: ``GROUP BY grade HAVING count(*) > 500`` — you can't write that condition as a WHERE, because count(*) doesn't exist until after grouping.

Q8. Explain the different types of JOINS, with an example.

INNER JOIN returns only rows that have a match in both tables — if a row on either side has no match, it's dropped entirely. LEFT JOIN returns every row from the left (first) table regardless of whether it finds a match on the right — unmatched columns from the right table just come back as NULL. RIGHT JOIN is the mirror image of LEFT JOIN. FULL JOIN keeps everything from both sides. In this project, ``mart_loan_el`` uses ``LEFT JOIN lgd_by_grade`` — every loan keeps its row even if its grade doesn't have a specific LGD estimate; it just falls back to a default value via COALESCE instead of the whole loan disappearing from the result, which is exactly what would happen with an INNER JOIN.

Q9. What does COALESCE do? Give an example.

COALESCE takes a list of values and returns the first one that isn't NULL — it's a clean way to provide a fallback/default value. Real example from this project: ``COALESCE(g.lgd, ${lgd})`` — use the grade-specific loss estimate if one exists for that loan's grade, otherwise fall back to the overall portfolio-wide figure.

Q10. What's a CTE (the WITH clause), and why use one instead of a subquery?

A CTE (Common Table Expression) lets you name an intermediate result and reference it later in the same query, which makes multi-step logic much more readable than nesting subqueries inside each other. This project's portfolio-summary query starts with ``WITH banded AS (SELECT *, CASE WHEN pd_hat < 0.05 THEN 'Low' ... END AS risk_band FROM mart_simulator_base)`` — first tag every loan with a risk band, give that intermediate result a name, then GROUP BY on top of it. Written as nested subqueries instead, that would be much harder to read.

Q11. What's a window function? Have you used one?

A window function runs a calculation across a set of related rows WITHOUT collapsing them into a single row the way GROUP BY does — each original row stays, with the calculated value attached. This project uses ``QUALIFY row_number() OVER (PARTITION BY loan_id ORDER BY issue_d) = 1`` during cleaning: for each loan_id, number its records ordered by issue date, and keep only the first one — a clean way to de-duplicate while choosing which duplicate to keep, which a plain GROUP BY / DISTINCT couldn't do as precisely.

Q12. Walk me through a real GROUP BY query from your project.

The portfolio-summary aggregation groups loans by grade, by loan purpose, by vintage year, and by risk band, computing loan count, total funded amount, average predicted default probability, and total expected loss for each group — the exact numbers the dashboard's segment breakdowns are built from. Being able to say this sentence fluently, from memory, about your own project matters more than reciting GROUP BY syntax.

Q13. What's the difference between a primary key and a foreign key?

A primary key uniquely identifies each row within its own table — no two rows can share one, and it can't be NULL. A foreign key is a column in one table that references a primary key in another table, creating a link between them. In this project, `loan_id` is the primary key of the loan tables, and it appears as the linking key across the fact and dimension tables in the star schema.

Q14. DISTINCT vs GROUP BY — aren't they the same thing?

They overlap but aren't the same. DISTINCT simply removes duplicate rows from a result with no aggregation involved — you can use it on its own, like `SELECT DISTINCT issue_d FROM mart_loan_features` to list each unique date once. GROUP BY exists specifically to let you compute aggregates (SUM, COUNT, AVG) per group. If you're not aggregating anything, DISTINCT is simpler and clearer.`

Q15. What's normalization? Is your data normalized?

Normalization is organizing tables to reduce duplicated data — splitting information into separate related tables instead of repeating it in one wide table. This project deliberately does a bit of both: the cleaning/feature layer keeps things fairly normalized (loan info, benchmark info, and outcome info live in separate tables), while the BI layer (the star schema) intentionally DE-normalizes into a fact table plus a few dimension tables, because that's the standard, fast-to-query shape BI tools like Power BI expect.

SECTION 3

Excel / Power BI / dashboards

Very commonly asked even when the project is code-heavy, because most Indian analyst roles still use Excel and Power BI daily.

Q16. What's a star schema? Did you actually build one?

A star schema is a way of organizing a BI data model around one central FACT table (the numbers you want to measure and aggregate) surrounded by smaller DIMENSION tables (the descriptive attributes you slice those numbers by) — named for the shape it makes in a diagram, one fact table in the middle with dimensions radiating out. Yes — this project has one: FACT_LOAN at the center (loan amount, predicted default probability, expected loss, one row per loan), surrounded by DIM_DATE (when it was issued), DIM_BORROWER (income band, home ownership, state), and DIM_LOAN_PRODUCT (purpose and grade).

Q17. What's the difference between a fact table and a dimension table?

A fact table holds the measurable, numeric events you want to analyze — things you SUM, AVERAGE, or COUNT, like expected loss or loan amount. A dimension table holds descriptive context you use to slice and filter those facts — things like date, category, or region, which you group by rather than sum. In this project, FACT_LOAN is where every loan's numbers live; DIM_BORROWER, DIM_DATE, and DIM_LOAN_PRODUCT are what let you ask 'expected loss by income band' or 'by month' without repeating that descriptive information on every single row.

Q18. What's DAX, and how comfortable are you with it?

DAX (Data Analysis Expressions) is Power BI's formula language, used to write calculated columns and measures — for example a measure like `Total Expected Loss = SUM(FACT_LOAN[expected_loss])``, or something more advanced using functions like CALCULATE to change filter context. Be honest here about your actual depth — if your DAX is basic, say so plainly and pivot to what you DO know well (the schema design, or the SQL underneath it). A confident, honest 'I'm still building depth in DAX specifically, here's what I've used so far' lands far better than getting caught bluffing on a follow-up question.

Q19. What makes a dashboard good, versus just a pile of charts?

Pull this straight from your own design choices: lead with a small number of the figures that matter most (this project's dashboard opens with exposure, default rate, high-risk accounts, and expected loss — four or five numbers, not twenty), have one dominant visual that answers the single most important question at a glance (the risk-distribution bar answering 'how risky is the book right now'), and be explicit about what's a real measured number versus an illustrative one — this project visibly badges every panel as REAL or DEMO data for exactly that reason, so a viewer never mistakes a placeholder for a fact.

Q20. What's the Excel equivalent of a SQL JOIN?

VLOOKUP or INDEX-MATCH — both look up a value in one table based on a matching key and pull back a related value from another table, which is the same underlying idea as a JOIN. The differences: VLOOKUP can only look rightward and returns one match; a SQL JOIN has no such direction limit and naturally returns every matching row, not just the first one — which matters if a key isn't unique.

Q21. Have you used pivot tables? How do they relate to GROUP BY?

A pivot table in Excel does the same conceptual job as a SQL GROUP BY plus aggregation — pick a field to group by (rows), a field to aggregate (values, e.g. sum of expected loss), and optionally a field to break results out by column. If you've built any of the segment breakdowns in this project by hand while exploring the data, mention that concretely rather than just saying yes/no.

Python & pandas

Kept deliberately basic here — a fresher Data Analyst round tests whether you can read and reason about pandas code, not whether you can implement a model from scratch.

Q22. What is a pandas DataFrame?

A 2-dimensional, labelled data structure in Python — rows and named columns, conceptually the same as a spreadsheet or a SQL table, but manipulable directly in code. Most of this project's data-cleaning and model-training logic operates on DataFrames pulled straight out of SQL query results.

Q23. How do you handle missing values in pandas?

Common tools: `.isnull()` / `.isna()` to detect them, `.dropna()` to remove rows or columns containing them, and `.fillna()` to fill them with a chosen value (a constant, the column mean/median, or a forward/backward fill). This project handles missing values inside a scikit-learn preprocessing pipeline rather than by hand: numeric fields get filled with the column's median, categorical fields get filled with the most frequent value — and critically, that fill logic is fit once on the training data and then applied identically to the test data, so there's no risk of the test set's own values leaking into how missing values get filled.

Q24. What's the difference between a Python list and a NumPy array?

A list can hold mixed data types and isn't optimized for numerical work — looping over it in Python is relatively slow. A NumPy array holds a single fixed data type and supports vectorized operations, meaning a calculation applies to the whole array at once using fast, compiled code underneath, instead of an explicit Python-level loop over every element.

Q25. What does `.groupby()` do in pandas, and how does it relate to SQL?

The same underlying idea as SQL's GROUP BY: split the DataFrame into groups based on one or more column values, apply an aggregation function to each group (sum, mean, count, etc.), then combine the results back into a single output. If you can explain one SQL GROUP BY query from your project fluently, you can explain `.groupby()` the same way — same concept, different syntax.

Q26. What's the difference between `.loc[]` and `.iloc[]` in pandas?

`.loc[]` selects rows and columns by their LABEL (the actual index value or column name). `.iloc[]` selects by INTEGER POSITION, regardless of what the labels actually are — the same way you'd index into a plain Python list. `.loc[5]` gets the row labelled 5; `.iloc[5]` gets the 6th row (position 5, zero-indexed), whatever its label happens to be.

SECTION 5

Statistics fundamentals

Expect 2-4 of these. The bar for a fresher is explaining the concept simply and correctly, in your own words, ideally with a project example — not deriving a formula on a whiteboard.

Q27. Mean vs median — when would you use one over the other?

The mean (average) is pulled around by extreme values; the median (the middle value when sorted) isn't. When a distribution is skewed — a few very large values dragging the average up — the median describes the 'typical' case better. Loan amounts and annual income in this project are right-skewed (most borrowers cluster at moderate values, but a smaller number of high earners stretch the distribution upward), which is exactly why the pipeline uses median imputation for missing numeric fields instead of filling with the mean — the median is a more honest 'typical value' here.

Q28. What's an outlier, and how would you handle one?

A value that falls far outside the normal range of the rest of the data — sometimes a genuine, real extreme case, and sometimes a data-entry or measurement error. You generally shouldn't just delete outliers blindly, because some are real and meaningful. This project's cleaning stage instead BOUNDS clearly impossible values rather than deleting rows outright — for example, ``dti BETWEEN -1 AND 1000`` catches values that couldn't be a genuine debt-to-income ratio, while still keeping legitimately high (but plausible) values in the data.

Q29. Correlation vs causation — explain with an example.

Two variables moving together (correlation) doesn't prove one causes the other — there could be a third factor driving both, or the relationship could run the opposite direction, or it could simply be coincidence in this particular sample. Real example from this project: lower-income applicants were found to be approved at a noticeably lower rate than higher-income ones. That's a real, measured correlation in the approval data — but on its own it doesn't prove income directly CAUSES the gap; a proper investigation would need to check for other factors moving alongside income (like credit history length or loan purpose) before concluding anything about cause.

Q30. What's a p-value, in plain language?

Roughly: if there were truly NO real effect or difference, how surprising would the result you actually observed be, just from random chance alone. A small p-value means your result would be quite surprising under 'nothing is really going on' — so it's taken as evidence something real is happening. This project used exactly this idea to check whether the model's improvement over a simple baseline was a real, reliable effect or could plausibly have happened by chance on this particular sample of loans.

Q31. What's the difference between a population and a sample?

The population is the entire group you actually care about; a sample is the smaller subset you have real data for and draw conclusions from. This project's 640,000+ loans are a SAMPLE of one lender's historical lending activity over a few years — not the full population of every loan ever made anywhere, which is worth being able to say plainly if asked how far the findings generalize.

Q32. What's standard deviation telling you, intuitively?

How spread out the values in a dataset are around the average — a small standard deviation means most values sit close to the mean; a large one means they're scattered widely. It's the same basic idea used, at a more advanced level, in this project's population-stability check: comparing how spread out and positioned a feature's values are between two different time periods, to see whether the underlying population has shifted.

The honest, self-aware questions

Freshers underestimate these, but they often matter more than the technical ones — they're testing judgment and honesty, not just knowledge. Prepare a real, specific answer for each; a vague or overconfident one is worse than admitting a limitation clearly.

Q33. What's a limitation of your project that you're aware of?

Have one specific, real answer ready rather than something generic. A strong example from this project: the expected-loss figure uses each loan's ORIGINAL funded amount as the exposure at risk, not how much of it is actually still outstanding — so for a loan that's already had two years of payments made against it, the model is overstating how much money is genuinely still at risk. Naming a real, specific limitation like this — and being able to say what the proper fix would be — reads as far more credible than claiming the project has no weaknesses.

Q34. If this were a real bank's system, what would be missing before it could go live?

Three honest gaps worth naming: live monitoring against real, continuously-updating data (this project can only compare two already-existing historical snapshots, not watch a live stream); a proper way to account for the fact that this data only contains loans that were actually APPROVED historically, with no information about what would have happened to people who were rejected; and independent review/sign-off from a model-validation team before anyone trusts it with real money.

Q35. How do you know your data is actually clean, rather than just assuming it?

Point to concrete, specific checks rather than a general 'I cleaned it' claim: logging null rates and row counts at each stage of the pipeline so a problem would be visible, filtering out loans that hadn't reached a genuinely final outcome yet before including them in modelling, removing duplicate records by loan ID, and bounding fields like debt-to-income to a plausible range instead of trusting every raw value blindly.

Q36. Did you validate your model, or just trust whatever accuracy number came out?

This is one of your strongest answers if you say it clearly: the model was tested on loans issued AFTER the period it was trained on, not on a random split of the same time period — which matters because in the real world a model only ever scores loans it hasn't seen yet, from the future relative to its training data. On top of that, the model was compared against a simple existing baseline to check it was actually adding real value, not just producing an impressive-looking number that happened to fit this particular dataset well.

Q37. What's one mistake you made while building this, and how did you catch it?

Have a real one ready — interviewers respond well to genuine specifics here. A good template: name a number or calculation that was wrong at some point (even a small one), explain briefly how you noticed it (a sanity check, a number that looked implausible, re-deriving it a different way and getting a different answer), and what you changed as a result. Being able to describe your own debugging process honestly is a stronger signal than pretending everything worked on the first try.